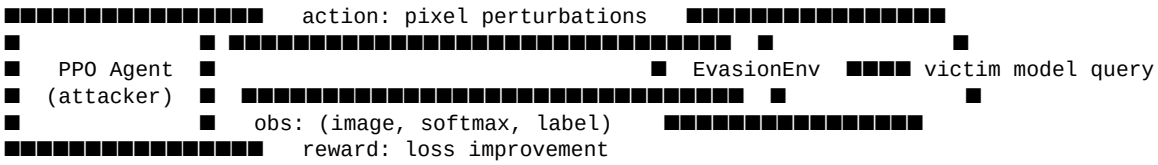
How the RL Attack Works in This Codebase

A concrete, code-level walkthrough of the adversarial-agents pipeline.

---

1. The Big Picture

The goal is to train an RL agent that learns to **perturb images** so a victim classifier misclassifies them. The agent doesn't know the victim's weights (black-box); it only sees the victim's softmax output after each query.



One "episode" = one image. The agent repeatedly perturbs it and queries the victim until the victim misclassifies (success) or 1000 queries are exhausted (failure).

---

2. The Environment (`environment.py` → `EvasionEnv`)

2.1 State Space (what the agent sees)

A dictionary with three keys, defined at line 45:

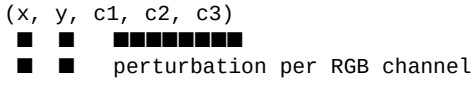
Key	Shape	Contents
`image`	`(3, 224, 224)`	The current perturbed image, **after** preprocessing (resize 256 → center-crop 224 → ImageNet normalize). This is what the victim model would see.
`outputs`	`(10,)`	The victim's softmax probability vector `Z(x_t)` from the last query.
`source_class`	discrete(10)	The true label `y` of the original image (one-hot encoded by the extractor).

The raw image `self.sample` is always stored at its original resolution (32×32 for CIFAR-10). Preprocessing is applied on-the-fly in `get_obs()` and `dynamics()`.

2.2 Action Space (what the agent outputs)

A continuous box of shape `(N * 5,)` in `[-2, 2]`, defined at line 56.

`N` is the number of pixels to perturb per step (default 5). Each pixel gets 5 values:



```
■  ■■■ pixel row
■■■■■■ pixel column
```

Coordinate mapping (line 70–75): the raw action value `a ∈ [-2, 2]` is mapped to a pixel index via:

```
pixel_index = min(int((a + 2.0) * 32 / 4.0), 31)
```

This linearly maps `[-2, 2]` → `[0, 31]` for 32×32 images.

Perturbation magnitude (line 77–79): each channel perturbation is:

```
Δ = action_value * θ / 2
```

where `θ` (theta) defaults to 0.05. So the max single-step per-pixel perturbation per channel is `2.0 * 0.05 / 2.0 = 0.05`. The result is clamped to `[0, 1]`.

2.3 The `generate_query()` Method (line 65)

This is where actions turn into perturbed images:

1. For each of the `N` pixel groups in the action vector, decode `(x, y)` → pixel coordinates and `(c1, c2, c3)` → per-channel deltas.
2. Apply each delta to `self.sample[c, x, y]`, clamped to `[0, 1]`.
3. **RLMaxLoss only**: if the total L2 perturbation `■x_perturbed - x_original■■` exceeds `ε`, project the perturbation back onto the ε-ball:

```
δ = (δ / ■δ■■) * ε
x_perturbed = x_original + δ
```

2.4 The `dynamics()` Method (line 96)

Queries the victim model and computes the attack loss:

1. Preprocess the 32×32 image → 224×224 ImageNet-normalized tensor.
2. Forward pass through the victim → raw logits.
3. Apply softmax if outputs don't already sum to 1.
4. Compute `f(x, y) = -log(confidence_y + 1e-10)` — the negative log-confidence on the true class. Higher = victim is less sure about the correct class = attack is working.

2.5 Reward Function (line 127–137)

RLMaxLoss (the default):

```
reward = 20 * (f_current - f_previous)
```

The reward is the **improvement** in attack loss, scaled by 20×. If the victim became less confident in the true class, reward > 0. If more confident, reward < 0.

RLMinNorm (alternative):

```
reward = c * (prev_distortion - current_distortion) + 20 * (f_current - f_previous)
```

Adds a term that rewards reducing the L2 perturbation norm (find a smaller perturbation that still fools the victim).

2.6 Hill Climbing Transition (line 140–147)

This is a critical mechanism. **If reward < 0** (the action made things worse):

- The reward is overwritten to **0.0**
- The image is **reverted** to the previous best state (`self.s_tminus1`)
- The internal loss/distortion trackers are NOT updated

If reward ≥ 0 (improvement or no change):

- The new state is accepted
- `self.model_loss`, `self.cumulative_lp`, and `self.s_tminus1` are all updated

This means the environment **never goes backward**. It's a monotonic ratchet: the agent can only maintain or improve its attack progress. Bad actions are "free" (reward = 0, state unchanged). This makes the reward landscape much easier for PPO to learn from — no negative rewards, no catastrophic regressions mid-episode.

2.7 Episode Termination (line 149–159)

- **Done** (`done=True`): the victim's argmax prediction ≠ the true label. The attack succeeded.
- **Truncated** (`truncated=True`): 1000 queries reached without misclassification. The attack failed (ran out of budget).

2.8 Episode Reset (line 208)

1. Randomly sample an image from the dataset.
2. Query the victim on the **unperturbed** image (with preprocessing).
3. If the victim classifies it correctly, use it. Otherwise, sample again (skip images the victim already gets wrong — no point attacking those).
4. Initialize all trackers: loss, distortion, reward accumulators, etc.

---

3. The PPO Agent (`ppo_agent.py` ⟶ `PPOAgent`)

3.1 Feature Extractor: `EfficientNet` (line 26)

A custom SB3 `BaseFeaturesExtractor` that processes the dict observation:

Observation key	Network	Output dim
`image`	EfficientNet-B0 (random init) → Linear(1280→512) → ReLU → Flatten	512
`outputs`	Linear(10→10)	10
`source_class`	Linear(10→10)	10

All three are concatenated → **532-dimensional** feature vector.

Key detail: the EfficientNet is initialized with `weights=None` (random weights), not pretrained. It learns its image features from scratch during PPO training.

3.2 Policy and Value Heads

Both are 2-layer MLPs with ReLU activation:

```
features (532) → Linear(256) → ReLU → Linear(256) → ReLU → output
```

- **Policy head** (`pi`): outputs a Gaussian distribution over the `N*5`-dimensional action space (mean + log-std).
- **Value head** (`vf`): outputs a single scalar V(s) for advantage estimation.

3.3 Full Forward Pass Summary

```
32×32 image ■■■ preprocess ■■■ 224×224 ■■■ EfficientNet-B0 ■■■ 512-d
                                                                ■
victim softmax (10,) ■■■■■■■■■■■■ Linear ■■■■■■■■■■■■■■■■■■■■■■■■ 10-d ■■■ concat (532) ■■■ MLP ■■■ ac
                                                                ■
true label one-hot (10,) ■■■■■■■■ Linear ■■■■■■■■■■■■■■■■■■■■■■ 10-d
```

---

4. Training Loop (`train_agent.py`)

4.1 Data Split

The CIFAR-10 test set (10,000 images) is split 50/50:

- **D** (first 5,000): used for RL training
- **D'** (last 5,000): used for evaluation

4.2 PPO Hyperparameters

Parameter	Value	Meaning
`n_envs`	32 (default)	Parallel environment copies
`n_steps`	128	Steps collected per env before each PPO update
`batch_size`	64	Minibatch size for PPO gradient updates
`n_epochs`	10	Number of passes over the rollout buffer per update
`gamma`	0.99	Discount factor
`learning_rate`	`λ(x) = x * 0.0025`	Linear decay schedule (x goes from 1→0 over training)
`clip_range`	`λ(x) = x * 0.1`	Linear decay schedule for PPO clipping
`training_steps`	4,915,200	Total env steps = 1200 updates × 128 steps × 32 envs

4.3 One PPO Update Cycle

1. **Rollout**: each of the 32 envs runs 128 steps, collecting `(obs, action, reward, done)` tuples → 4096 transitions total.

2. **Advantage estimation**: GAE(λ) computes advantages using the value head predictions across the rollout.

3. **Policy optimization**: 10 epochs over the 4096 transitions in minibatches of 64, updating policy and value networks with the clipped PPO objective.

4. Repeat 1200 times.

4.4 What Happens Inside Each Env Step

```
Agent outputs action (N*5 floats)
    ■
    ▼
generate_query(): apply pixel perturbations to 32×32 image, project if needed
    ■
    ▼
dynamics(): preprocess → victim forward pass → softmax → compute f(x,y)
    ■
    ▼
reward = 20 * (f_new - f_old)
    ■
    ▼
if reward &lt; 0: revert image, set reward=0  (hill climbing)
if victim misclassifies: episode done (success)
if queries ≥ 1000: episode truncated (failure)
    ■
    ▼
return (preprocessed_image, softmax_vec, true_label), reward, done
```

---

5. Evaluation (`evaluate_agent.py`)

Deterministic rollout over every image in D' (the held-out 5000):

1. Load the trained policy weights from `agents/.pth`.

2. For each image in D': reset env with that specific image (`setImage()`), run the agent until done or truncated.

3. Log per-episode results to `evaluation_results/episodic.csv`: queries used, start/end loss, reward, label, prediction, distortion.

The key metrics from the episodic log:

- **Attack success rate**: fraction of episodes where `prediction ≠ label` (done=True)
- **Queries per attack**: how many victim model calls it took
- **Distortion**: L2 norm of the perturbation

---

6. Two Attack Modes

RLMaxLoss (default)

- **Goal**: maximize victim's loss on the true class within a fixed perturbation budget ε.
- **Constraint**: perturbations projected onto the L2 ε-ball after each step.
- **Reward**: purely loss improvement (distortion is constrained, not optimized).

RLMinNorm

- **Goal**: find the smallest perturbation that causes misclassification.
- **No fixed ε budget**: perturbations can grow freely.
- **Reward**: loss improvement + bonus for shrinking the perturbation norm, weighted by `c`.

---

7. Key Design Choices and Their Implications

1. **Hill climbing (reward ≥ 0 only)**: the agent never receives negative signal. This simplifies learning but means the policy gradient only distinguishes between "no progress" (r=0) and "some progress" (r>0). The agent cannot learn from bad actions — it only learns which directions are productive.

2. **Image stored at 32×32, observed at 224×224**: the agent perturbs at native resolution but sees the preprocessed version. This means the agent's EfficientNet feature extractor processes images at a different resolution than the perturbation grid.

3. **EfficientNet from scratch**: the image branch starts random, so early training produces near-random features. The `outputs` (victim softmax) and `source_class` branches are likely the dominant signal early on.

4. **One victim query per step**: the environment calls `dynamics()` once per `step()`. With up to 1000 steps per episode and 32 parallel envs, a single PPO rollout of 128 steps potentially makes `128 * 32 = 4096` victim queries. The victim forward pass is the main computational bottleneck.

5. **20× reward scaling**: raw log-confidence changes between steps are tiny (e.g., 0.001). The 20× multiplier makes the reward signal large enough for PPO to learn from, relative to the value function's scale.